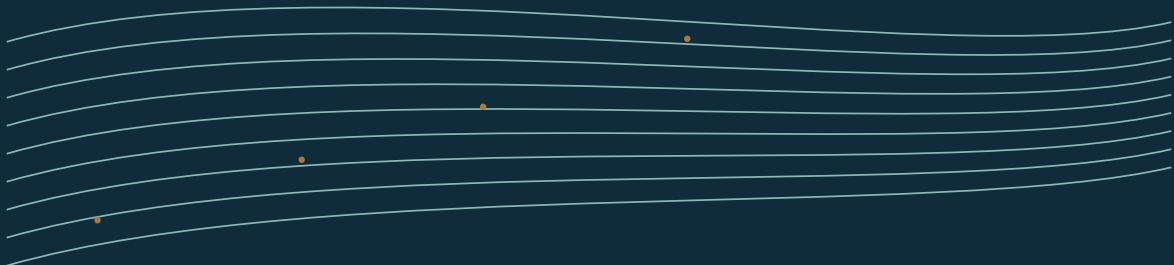

ALPHALAB

Master Architecture

KNOWLEDGE-TO-CAPITAL OPERATING SYSTEM

A clear, governed architecture for transforming market knowledge into deterministic context, complete trading policies, protected evidence, controlled capital, secure execution and assumption-aware learning.



Document Control and Reading Guide

Purpose

This report is the English architecture reference for the Alpha Lab Obsidian master-architecture vault. It defines the system-level flow, authority boundaries, permanent shared services, context-specific responsibilities, evidence standards and delivery sequence. It is an architecture document, not a trading recommendation and not a claim that any strategy has been validated.

Document	Alpha Lab Master Architecture
Edition	Clear Reference Edition 2.0
Canonical source	Alpha Lab Obsidian Master Architecture Vault
Language	English
Status	System architecture and implementation reference
Audience	Quant researchers, market-domain owners, ML engineers, MQL5/Python developers, risk and deployment owners
Scope	Knowledge, context, setup discovery, research integrity, evidence, capital, portfolio, runtime, execution, monitoring and research memory

Reading Sequence

Read the report in order the first time. Later chapters assume the contracts and authority boundaries established earlier. During implementation, use the contents page to jump directly to the layer being designed or reviewed.

The interpretation rule

Context states what the market was and when that fact became knowable. A setup states how the system proposes to exploit that context. Evidence states whether that exploitation is defensible. Capital states how much risk may be allocated. Execution states how the approved decision is safely expressed. Monitoring states whether the assumptions that justified the edge remain alive.

Architecture at a Glance

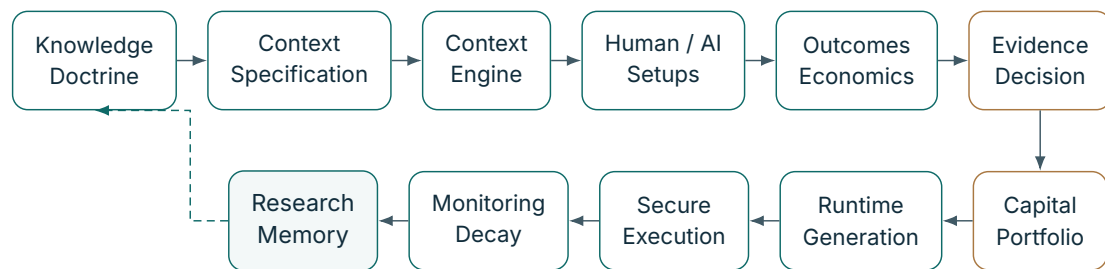


Figure 1: Canonical knowledge-to-capital transformation and governed learning loop.

The architectural thesis

Solve repeated mechanisms once. Preserve context-specific meaning explicitly. Require human and AI setup branches to carry the same economic and evidential burden. Keep capital authority independent. Preserve every success, failure and live incident as governed research memory.

Six Governing Laws

-
- | | | |
|-----------|--------------------------------------|--|
| 01 | Meaning before implementation | Market ontology and known-time semantics are fixed before code and profitability pressure. |
| 02 | Context is not a trade | Observation, exploitation, proof and capital authority remain separate. |
| 03 | Search is constrained | Human and AI branches share causal economics, trial accounting and evidence burden. |
| 04 | Evidence is staged | No single backtest, model score or narrative grants production authority. |
| 05 | Capital is independent | Strategy and model layers cannot self-approve exposure. |
| 06 | Failure compounds | Rejected trials, incidents and edge decay become governed research memory. |

Contents

Document Control and Reading Guide	i
Architecture at a Glance	iii
Executive Summary	vi
I Foundations	1
1 Canonical End-to-End Architecture	2
2 Knowledge-to-Context Architecture	7
II The Context Factory	11
3 Context Factory Architecture	12
III Setup Engineering and AI Discovery	16
4 Two-Branch Setup Architecture	17
5 AI Setup Discovery Architecture	21
IV Research Integrity and Evidence	24
6 Research Firewall and Evidence	25
7 Statistics, Monte Carlo and AI Analyst	28
V Capital, Production and Live Control	31
8 Capital and Portfolio Architecture	32
9 Runtime, Security and Execution	35
10 Assumption-Aware Monitoring	39
VI Scale, Governance and Delivery	42
11 Compounding Speed and Scalability	43
12 Authority, Lineage and Artifact Chain	47
13 Implementation Program	50
14 Architectural Risks and Controls	54
A Acceptance Gates and Completion Criteria	59
B Canonical Object Catalogue	62
C Platform KPI Catalogue	64
D Source Library Map	66
E Glossary	69

Executive Summary

Alpha Lab is not a single strategy and it is not merely a backtesting framework. It is a governed operating architecture for converting a human market worldview into deterministic context, complete exploitation policies, protected evidence, controlled capital, secure execution and live assumption monitoring.

Its defining choice is separation of authority. Doctrine owns market meaning. Context engines own occurrence identity and lifecycle. Human or AI setup branches construct complete treatments. The research system produces causal and economic outcomes. A deterministic evidence gate owns promotion eligibility. Capital and portfolio governance own exposure. The execution layer owns broker interaction after a local hard-risk gate. Monitoring owns health state, reduction, quarantine and retirement recommendations.

Executive thesis

Alpha Lab creates compounding advantage by solving repeated mechanisms once, preserving context-specific meaning explicitly and converting every successful or failed experiment into reusable infrastructure, data and research intelligence.

The Two Setup Branches

After a context has passed semantic and causal conformance, the architecture opens two equal branches. A human-authored setup begins with a falsifiable thesis, detailed setup document, implementation design and complete treatment policy. The AI branch receives immutable context packages, governed representations, a compatible treatment universe, objective profiles and a protected experiment budget. Both branches must be able to abstain, both produce the same canonical treatment artifact, and both enter the same outcome, economics, statistical and prospective evidence system.

Evidence Before Capital

The architecture treats a locked final test as a consumable asset. All protected access is registered. Every considered model, treatment and threshold remains in the trial ledger. Statistical support requires cluster-aware inference, Monte Carlo, false-discovery controls, matched nulls, cost and execution stress, parameter and regime stability, and prospective evidence. An AI Analyst can explain registered artifacts and propose falsification tests, but it cannot promote a policy by narrative.

The Closed Learning Loop

Live execution distinguishes expected decision, expected fill, actual request, actual fill and actual position. Monitoring compares real behavior with the assumptions that justified promotion. Drift may move a policy from Healthy to Watch, Reduced, Quarantined or Retired. Live data can initiate a new governed experiment, but it cannot silently rewrite doctrine or production.

Why the Model Compounds

Each new context inherits shared market truth, identity, lifecycle, treatment construction, economics, statistics, runtime and monitoring services. Each completed experiment adds treatment outcomes, failure knowledge and live assumptions to Research Memory. The long-run advantage is therefore not merely faster implementation; it is a permanently decreasing marginal cost of trustworthy research and a progressively better prior for the next experiment.

PART I

Foundations



Canonical End-to-End Architecture

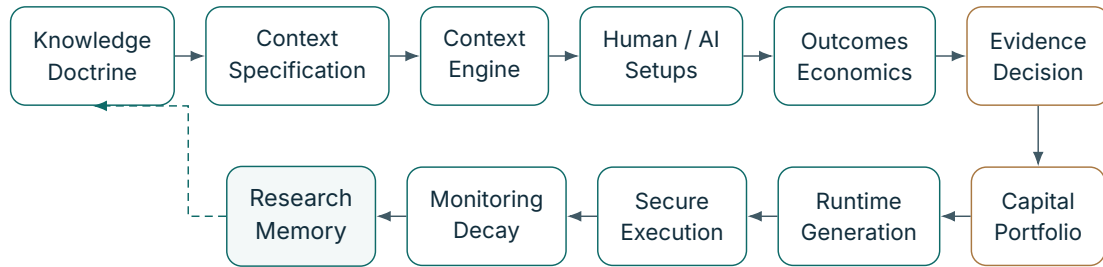


Figure 1.1: Canonical knowledge-to-capital transformation and governed learning loop.

1.1 Architectural Mission

Alpha Lab must transform an informal market viewpoint into a capital-bearing live policy without allowing any stage to silently take authority from another stage. The pipeline is therefore built as a sequence of governed transformations, each with its own inputs, outputs, owners, evidence and failure modes.

```

Human market knowledge
→ formal doctrine
→ context specification
→ deterministic context engine
→ manual or AI exploitation policies
→ causal/economic outcomes
→ statistical evidence
→ capital and portfolio policy
→ immutable runtime generation
→ secure execution
→ assumption-aware monitoring
→ research memory
  
```

1.2 The Ten Planes

1.2.1 Knowledge Plane

Captures the human model of the market: ontology, vocabulary, causal story, examples, counterexamples, ambiguity, known-time and lifecycle. It defines meaning but does not decide trades.

1.2.2 Context Plane

Converts market truth into immutable context occurrences. It answers: what happened, when was it knowable, what state is it in, and what identity does it have?

1.2.3 Setup Plane

Defines exploitation. A setup may be written by a human or discovered by AI, but it must be a complete treatment: eligibility, entry, stop, exit, management, sizing constraints and abstention.

1.2.4 Research Plane

Generates path outcomes, executable economics, datasets, models, baselines and experiments. It is allowed to search, but only inside an explicit trial budget and protected data-role system.

1.2.5 Evidence Plane

Determines whether an observed result survives uncertainty, multiple testing, resampling, stress, null comparisons, out-of-sample tests and prospective evidence.

1.2.6 Capital Plane

Tests how a supported edge should be sized. It cannot turn a weak or negative edge into a valid one.

1.2.7 Portfolio Plane

Combines multiple edges, resolves overlapping exposure, allocates risk, controls concentration and enforces portfolio-level loss limits.

1.2.8 Runtime Plane

Compiles approved context, feature, model, threshold, treatment, risk and monitoring artifacts into one immutable generation.

1.2.9 Execution Plane

Applies hard risk gates, broker preflight, idempotent order submission, fill reconciliation, position lifecycle and emergency controls.

1.2.10 Monitoring and Evolution Plane

Compares live market, model, execution, portfolio and performance behavior with the assumptions that justified promotion. It can recommend continue, reduce, quarantine, revalidate or retire.

1.3 Why the Separation Matters

A context may be semantically correct but economically useless. A setup may be poorly designed over an informative context. A model may be statistically promising but impossible to execute. An edge may remain valid while broker slippage destroys realized returns. A profitable strategy may become dangerous when combined with correlated strategies.

Without plane separation, all failures collapse into “the strategy stopped working.” With separation, the system can locate the failure and reuse every unaffected layer.

1.4 Shared Platform Versus Strategy-Specific Work

1.4.1 Shared permanently

- ♦ market truth and clock;
- ♦ identity and lineage;
- ♦ lifecycle framework;
- ♦ context SDK;
- ♦ treatment compiler;
- ♦ outcome and economics;

- ♦ dataset and split engine;
- ♦ trainer orchestration;
- ♦ statistical and anti-overfit suite;
- ♦ reporting and AI analyst;
- ♦ money-management library;
- ♦ portfolio allocation;
- ♦ runtime compiler;
- ♦ security, licensing and execution;
- ♦ monitoring, drift and retirement.

1.4.2 Strategy/context-specific

- ♦ doctrine and semantic meaning;
- ♦ unique anatomy extraction;
- ♦ context-specific states and fields;
- ♦ optional manual setup logic;
- ♦ treatment compatibility declarations;
- ♦ rare context-specific labels or execution semantics.

1.5 Evidence Is Not Binary

The system uses an evidence lifecycle:

```
Proposed
→ Context-Conformant
→ Research Candidate
→ Statistically Supported
→ Paper Validated
→ Micro-Live Validated
→ Production Qualified
→ Reduced
→ Quarantined
→ Retired
```

The phrase “this strategy has edge” is replaced with a conditional statement binding edge to context version, treatment, market, cost model, regime, horizon and evidence level.

1.6 The Closed Loop

The architecture is not complete until live evidence returns to research memory without contaminating current protected tests. Realized fills recalibrate economics. Drift creates new experiments. Failures become searchable knowledge. Retired edges remain available as negative evidence.

The loop must learn, but never mutate production invisibly.

Knowledge-to-Context Architecture

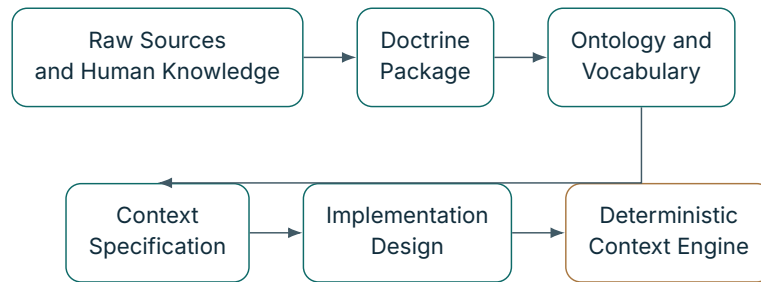


Figure 2.1: Meaning is frozen before implementation and profitability pressure can rewrite it.

2.1 Purpose

The first architectural responsibility is to preserve the human viewpoint before implementation or profitability pressure changes it. The system turns subjective understanding into a stable semantic asset.

2.2 Transformation Chain

```
Raw sources and human knowledge
→ doctrine package
→ ontology and vocabulary
→ ambiguity and assumption registers
→ formal context specification
→ implementation design
→ deterministic context engine
→ conformance evidence
```

2.3 Doctrine Package

Every context family starts with a versioned doctrine package containing:

- ♦ executive definition;
- ♦ market mechanism and rationale;
- ♦ exact vocabulary;
- ♦ entities, relationships and ownership;
- ♦ scope, non-goals and unsupported interpretations;
- ♦ event-time, known-time, confirmation-time and maturity-time;
- ♦ lifecycle states and terminal conditions;
- ♦ positive examples;
- ♦ visually similar negative examples;
- ♦ equality, overlap and one-tick boundary cases;
- ♦ invalidation, consumption and expiration;
- ♦ unresolved questions;
- ♦ source inventory and decision history.

Doctrine states what the market phenomenon means. It must not quietly include a preferred trade unless the trade action is itself part of the phenomenon.

2.4 Three Separate Documents

2.4.1 Doctrine Document

Owns semantic truth and examples.

2.4.2 Context Specification

Owns the machine-readable public contract: identity, fields, types, time semantics, lifecycle, compatibility, missingness and runtime requirements.

2.4.3 Implementation Design

Owns algorithms, state stores, synchronization, performance, persistence, replay, MQL5/Python responsibility, tests and phased delivery.

Keeping these separate prevents implementation convenience from rewriting meaning.

2.5 Canon Freeze

A context may move through:

Draft → Reviewable → Canonical → Superseded → Archived

After canon freeze, semantic changes require a new version and, when historical classification changes, a new research lineage. A definition cannot be edited after poor backtest results while pretending the same hypothesis was tested.

2.6 Human and AI Authority

Humans own ontology, unresolved interpretation, risk tolerance and canon approval. AI may extract, formalize, compare, critique, create fixtures, design algorithms and identify contradictions. AI cannot silently decide semantic truth.

2.7 Context Readiness

Before implementation:

- ♦ terms must have exact definitions;
- ♦ known-time must be explicit;
- ♦ state ownership must be unique;
- ♦ positive, negative and boundary cases must exist;
- ♦ missing/stale behavior must be declared;
- ♦ unresolved questions must be marked blocking or deferred;
- ♦ the intended context must be distinguishable from the intended setup.

2.8 Compounding Knowledge

Every solved ambiguity becomes reusable doctrine, fixture, primitive or rule. This is the first layer of compounding: the system no longer depends on remembering how the concept was understood during one conversation or one coding session.

PART II

The Context Factory

Context Factory Architecture

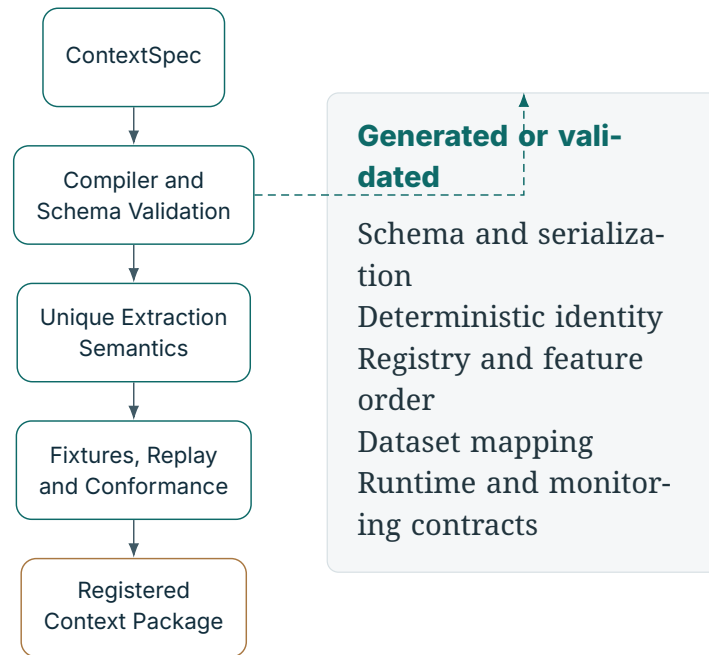


Figure 3.1: Declarative onboarding moves repeated mechanisms into the platform and leaves only unique market semantics hand-written.

3.1 Role

The Context Factory is the bridge between doctrine and every downstream strategy. It produces deterministic, causally valid and immutable context snapshots. It does not decide whether to trade.

3.2 Context Package

ContextPackage

```
|— DoctrineVersion
|— ContextSpec
|— AnatomyAdapter
|— FeaturePack
|— LifecycleProfile
|— RepresentationProfile
|— AllowedTreatmentFamilies
|— LabelProfile
|— MonitoringAssumptions
|— RuntimeCompatibility
|— ConformanceFixtures
```

3.3 Market Truth Layer

All context engines consume a canonical market layer rather than reimplementing broker/time logic. The shared layer owns:

- ♦ bars and ticks;
- ♦ bid/ask semantics;
- ♦ symbol specification and aliases;
- ♦ sessions, trading days and DST;
- ♦ higher-timeframe aggregation;
- ♦ multi-symbol synchronization;
- ♦ missing, stale, duplicated and reordered input states;
- ♦ source inventory and revisions;
- ♦ spread and executable-price observations.

Centralizing this layer removes a major class of repeated errors.

3.4 Universal Mechanisms, Specific Meaning

Reusable primitives may include structural points, references, boundaries, excursions, returns, confirmations, sequences, relationships, freshness, entitlement, expiration and invalidation.

The primitive supplies identity, lifecycle and replay mechanics. Each context still owns its exact semantic definition. The platform must not force

every theory into one universal definition of “break,” “pivot” or “confirmation.”

3.5 Context SDK and Compiler

The intended steady state is declarative onboarding. A ContextSpec declares required fields, state machine, known-time, identity, dependencies, compatibility and fixtures. The compiler generates or validates:

- ♦ schema and serialization;
- ♦ deterministic IDs;
- ♦ registry entry;
- ♦ feature-order contract;
- ♦ dataset mapping;
- ♦ test and fixture scaffolds;
- ♦ runtime compatibility;
- ♦ monitoring contract skeleton;
- ♦ documentation index.

The only hand-written core should be unique extraction semantics.

3.6 Context Occurrence Lifecycle

A generic form is:

```
Observed
→ Candidate
→ Confirmed
→ Eligible
→ Consumed / Invalidated / Expired
→ Retired
```

Each context can specialize states. The shared lifecycle framework validates transitions and records history, while doctrine defines the transition predicates.

3.7 Context Composition

Context occurrences may form graphs:

- ♦ parent/child;
- ♦ confirms;
- ♦ contradicts;
- ♦ prerequisite;
- ♦ higher-timeframe owner;
- ♦ intermarket relationship;
- ♦ temporal predecessor;
- ♦ shared opportunity cluster.

Combined contexts receive new identity and lineage; parent occurrences remain immutable.

3.8 Conformance Before Profitability

A context is research-enabled only after:

- ♦ semantic fixtures pass;
- ♦ future mutation cannot alter prior outputs;
- ♦ replay/restart preserve identity;
- ♦ lifecycle transitions are valid;
- ♦ historical and live modes agree;
- ♦ missing/stale behavior is correct;
- ♦ performance budgets are met;
- ♦ renderer and setup layers cannot mutate context truth.

A profitable but semantically wrong detector is rejected.

PART III

Setup Engineering and AI Discovery

Two-Branch Setup Architecture

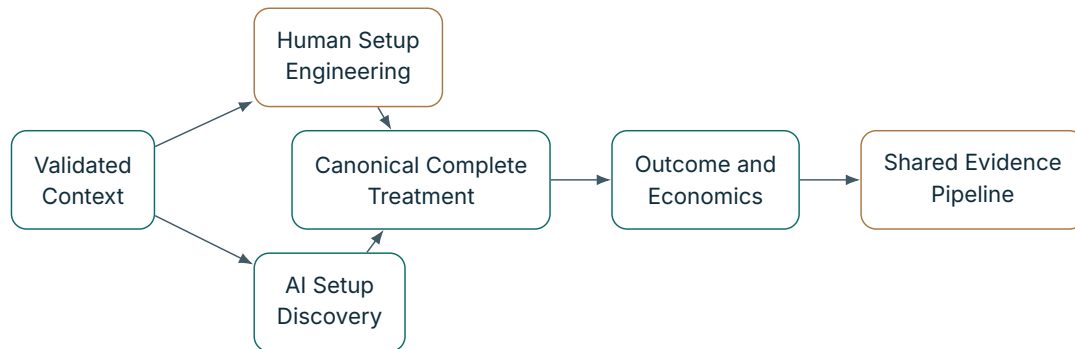


Figure 4.1: Human and AI branches differ in origin, not in the burden of proof.

4.1 The Split

After context validation, Alpha Lab opens two parallel but equal branches:

```

Context
├─ Human-authored setup branch
└─ AI setup-discovery branch
  
```

Both branches must produce the same canonical complete-treatment artifact and face the same research firewall, outcome engine, statistical tests and promotion gates.

4.2 Human Setup Branch

A human setup follows:

```
Setup thesis
→ detailed setup document
→ implementation design
→ complete treatment policy
→ scenario tests
→ central evidence pipeline
```

The setup document defines:

- ♦ target context versions;
- ♦ expected mechanism;
- ♦ eligibility and no-trade cases;
- ♦ direction;
- ♦ entry;
- ♦ stop;
- ♦ exit/target;
- ♦ trailing, partials and management;
- ♦ sizing constraints;
- ♦ economic assumptions;
- ♦ regime assumptions;
- ♦ falsification criteria.

Human conviction is valuable input but not evidence. A manual setup is not privileged over AI output.

4.3 AI Setup Branch

The AI branch receives context snapshots and a governed treatment universe. It may learn:

- ♦ trade versus skip;
- ♦ candidate ranking;
- ♦ treatment selection;
- ♦ expected utility;
- ♦ target/stop probabilities;
- ♦ survival or time-to-event;

- ♦ regime-specialist routing;
- ♦ uncertainty and novelty;
- ♦ post-entry management policy.

The AI must be able to return no trade.

4.4 Canonical Setup Artifact

A setup is not an entry trigger. It is:

```
Eligibility
× Entry
× Stop
× Exit
× Management
× Sizing Constraints
× Cost Profile
× Execution Profile
× Abstention Policy
```

The Treatment Compiler rejects incomplete, incoherent or broker-infeasible combinations.

4.5 Objective Styles

Styles such as high win rate, high reward, runner, low drawdown or high frequency are expressed as constrained multi-objective preference profiles. For example, high win rate is optimized only subject to minimum expectancy, sample, cost robustness and tail-loss limits.

4.6 Hybrid Policies

Human and AI decisions may be combined explicitly:

- ♦ human eligibility + AI skip/trade;
- ♦ human stop + AI exit;
- ♦ AI ranking + deterministic risk gate;
- ♦ human regime selector + specialist models.

The hybrid is represented as a versioned policy graph, not informal discretion.

4.7 Shared Research Destination

The origin of a setup is recorded for attribution, but once compiled, every setup enters the same trial ledger and evidence system. The architecture tests policies, not reputations.

AI Setup Discovery Architecture

5.1 Purpose

AI is used to discover how a defined context can be exploited, not to invent market meaning without boundaries. Human doctrine narrows the semantic world; AI searches the exploitation world.

5.2 Inputs

- ♦ immutable context package;
- ♦ fixed and versioned representations;
- ♦ allowed treatment atoms and compatibility graph;
- ♦ causal outcome cube;
- ♦ executable economic costs;
- ♦ data roles and folds;
- ♦ objective profile and hard guardrails;
- ♦ search budget;
- ♦ prior research memory permitted for this lineage.

5.3 Learning Tasks

The trainer system supports a ladder:

1. unconditional treatment baselines;
2. simple deterministic filters;
3. interpretable linear/probabilistic models;
4. tree and boosting models;

5. candidate ranking and treatment choice;
6. sequence, graph and multi-view models;
7. regime experts and ensembles;
8. constrained offline policy learning;
9. post-entry adaptation only after simpler tasks are proven.

Complexity must earn measurable, protected uplift.

5.4 Treatment Search

The search space is compiled and pruned before expensive training:

- ♦ semantic incompatibility;
- ♦ causality failure;
- ♦ invalid geometry;
- ♦ broker infeasibility;
- ♦ cost infeasibility;
- ♦ duplicate identity;
- ♦ deterministic dominance;
- ♦ insufficient sample or capability.

Every considered and pruned trial is registered for honest selection accounting.

5.5 Abstention and Uncertainty

AI output is not forced to trade. It can abstain because of stale context, novelty, high uncertainty, low expected utility, small candidate margin, excessive cost, portfolio conflict or runtime degradation.

Evaluation includes coverage, selective performance, calibration, missed opportunity and false-positive cost.

5.6 Search Governance

The trainer may not access locked final-test outcomes. Prompts, feature engineering, model selection and analyst feedback are all forms of exposure. Search budgets, objectives and stopping rules are compiled before protected evaluation.

5.7 Knowledge Output

The AI branch stores more than the winning model:

- ♦ treatments that consistently failed;
- ♦ feature families with no uplift;
- ♦ regime-specific strengths and weaknesses;
- ♦ cost and delay sensitivity;
- ♦ useful model families;
- ♦ uncertainty behavior;
- ♦ candidate relationships and dominance;
- ♦ explanations and proposed falsification experiments.

This knowledge becomes a prior for future contexts without replacing independent validation.

PART IV

Research Integrity and Evidence

Research Firewall and Evidence

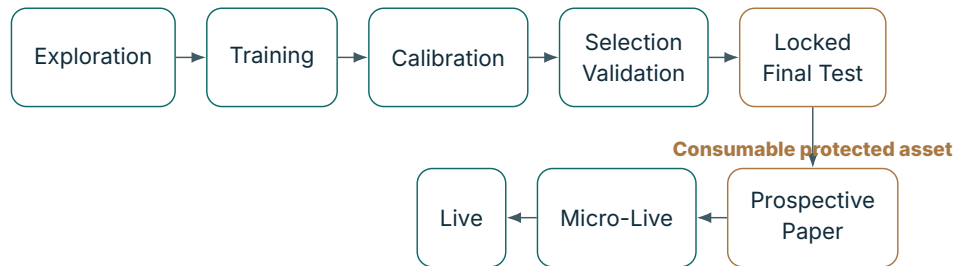


Figure 6.1: Protected data roles and exposure accounting prevent validation from silently becoming training.

6.1 Why the Firewall Is Central

A powerful AI search system can manufacture convincing false edges if it repeatedly sees validation and test results. The Research Firewall controls who and what may observe each data role.

6.2 Protected Data Roles

```

Exploration
→ Training
→ Calibration
→ Selection Validation
→ Locked Final Test
→ Prospective Paper
→ Micro-Live
→ Live
  
```

The final test is a consumable asset. Once viewed, summarized or used to alter the policy, it cannot remain pristine for the same lineage.

6.3 Exposure Ledger

Every protected access records:

- ♦ actor or process;
- ♦ experiment lineage;
- ♦ data and fields exposed;
- ♦ time and purpose;
- ♦ prompt or downstream decision;
- ♦ whether the dataset keeps its prior role.

AI-generated recommendations based on final-test outcomes count as exposure.

6.4 Experiment Compiler

Before a run, the compiler freezes:

- ♦ falsifiable hypothesis;
- ♦ context versions;
- ♦ treatment universe;
- ♦ outcome and label policy;
- ♦ cost profiles;
- ♦ data cutoff;
- ♦ fold and cluster plan;
- ♦ model ladder;
- ♦ objective and guardrails;
- ♦ trial budget and stopping;
- ♦ metrics and evidence criteria.

The compiler produces an immutable run plan and reproduction command.

6.5 Trial Ledger

Every attempt is retained:

- ♦ completed;
- ♦ failed;
- ♦ timed out;
- ♦ pruned;
- ♦ duplicate;
- ♦ invalid;
- ♦ cancelled.

Without the complete trial universe, multiple-testing and winner-selection adjustments are dishonest.

6.6 Evidence Lifecycle

Evidence moves through explicit states rather than pass/fail. Promotion depends on semantic correctness, net economics, uncertainty, false-discovery controls, stability, prospective parity, capacity and operational readiness.

6.7 Deterministic Gate Versus AI Analyst

The evidence gate applies machine-readable rules. The AI Analyst explains why profit or loss may have occurred and proposes new experiments. The analyst cannot promote a policy by narrative.

6.8 Failure as Output

Rejection is successful research when it is reproducible and its reason is retained. Failed contexts, treatments and models reduce future search cost through Failure Memory.

Statistics, Monte Carlo and AI Analyst

7.1 Central Statistical Engine

All manual and AI setups enter one shared statistical engine. Strategy authors cannot choose easier tests for preferred ideas.

The standard pack reports:

- ♦ independent opportunity and cluster counts;
- ♦ base rate and incremental uplift;
- ♦ expectancy and net utility;
- ♦ win rate and payoff distribution;
- ♦ confidence intervals;
- ♦ drawdown depth, duration and recovery;
- ♦ tail loss and expected shortfall;
- ♦ MFE, MAE and holding time;
- ♦ calibration, discrimination and coverage;
- ♦ regime, symbol, session and time buckets;
- ♦ cost and execution sensitivity;
- ♦ concentration in best trades, periods or clusters.

7.2 Dependence-Aware Inference

Trade rows are not automatically independent. Treatment siblings from one opportunity, overlapping holding periods and regime clusters require cluster/block resampling, purge and embargo.

7.3 Monte Carlo Engines

The central Monte Carlo layer includes:

- ♦ opportunity-cluster bootstrap;
- ♦ block or stationary bootstrap;
- ♦ trade-order/path uncertainty;
- ♦ spread, slippage and latency simulation;
- ♦ missed and partial fills;
- ♦ gap and tail scenarios;
- ♦ model and parameter uncertainty;
- ♦ portfolio dependence and ruin probability.

Outputs include terminal wealth distribution, drawdown, time underwater, risk-limit breach, worst quantile paths and capital requirements.

7.4 Anti-Overfit Suite

Mandatory controls include:

- ♦ nested/purged walk-forward;
- ♦ multiple-testing correction;
- ♦ deflated performance;
- ♦ probability of backtest overfitting;
- ♦ reality-check or superior-predictive-ability tests;
- ♦ parameter-surface stability;
- ♦ feature and treatment ablation;
- ♦ best-trade/period/cluster removal;
- ♦ cross-feed, cross-symbol and regime tests;
- ♦ cost, delay and execution stress;
- ♦ prospective paper evidence.

7.5 Null and Placebo Generator

Every context declares matched nulls preserving nuisance variables such as session, volatility, direction balance, holding period and cost. The platform can remove the context signal, shift timestamps, shuffle direction, substitute irrelevant pairs or compare against simple unconditional policies.

The key question is not merely “did it make money?” but “did this context add information beyond matched baselines?”

7.6 AI Analyst

The analyst receives registered artifacts and produces structured claims:

- ♦ profit source;
- ♦ loss source;
- ♦ regime/context attribution;
- ♦ treatment and feature attribution;
- ♦ cost/execution attribution;
- ♦ alternative explanations;
- ♦ confidence;
- ♦ counter-evidence;
- ♦ proposed falsification experiment.

All numeric claims must link to machine-generated artifacts. Unsupported explanations are labeled speculation.

7.7 Final Decision

Statistics and analysis create evidence. A deterministic gate and authorized human governance decide promotion. Profit alone never overrides leakage, non-reproducibility or critical execution failure.

PART V

Capital, Production and Live Control

Capital and Portfolio Architecture

8.1 Order of Operations

```
Support the edge
→ validate money-management policies
→ admit to portfolio
→ allocate capital
```

Money management cannot manufacture edge. Testing many sizing rules on the same return history is itself a new selection problem and requires separate validation.

8.2 Money Management Factory

Policy families may include:

- ♦ fixed cash risk;
- ♦ fixed fractional;
- ♦ stop-based maximum-loss sizing;
- ♦ volatility targeting;
- ♦ drawdown-responsive sizing;
- ♦ regime or opportunity-quality scaling;
- ♦ fractional Kelly with estimation penalty;
- ♦ risk tiers;
- ♦ portfolio-risk contribution targeting;
- ♦ equity and loss overlays.

The output is a versioned CapitalPolicy containing risk function, floors/ceilings, drawdown response, reserve requirement, stress behavior and kill conditions.

8.3 Capital Validation

Capital policies are evaluated under:

- ♦ cluster and block Monte Carlo;
- ♦ sequence and regime concentration;
- ♦ estimation error;
- ♦ tail gaps and adverse execution;
- ♦ delayed de-risking;
- ♦ parameter sensitivity;
- ♦ margin, lot step and broker constraints;
- ♦ portfolio interaction;
- ♦ time-under-water and recovery.

Selection targets robust utility, not maximum historical wealth.

8.4 Portfolio Engine

Valid strategies can still form an invalid portfolio. The portfolio layer owns:

- ♦ return and drawdown dependence;
- ♦ shared context/opportunity clusters;
- ♦ symbol, direction and factor exposure;
- ♦ simultaneous trade conflicts;
- ♦ capacity and liquidity;
- ♦ reservation ledger;
- ♦ concentration and portfolio kill limits;
- ♦ diversification value;
- ♦ strategy priority and scaling.

It may reject a valid individual trade because aggregate exposure is unacceptable.

8.5 Edge Genome

Every edge publishes a reusable profile:

- ♦ context family;
- ♦ market and horizon;
- ♦ direction and treatment style;
- ♦ return and tail shape;
- ♦ regime dependence;
- ♦ cost sensitivity;
- ♦ capacity;
- ♦ correlation and shared-event structure;
- ♦ evidence and decay state.

This allows the portfolio engine and research planner to identify duplicates, missing exposures and diversification opportunities.

8.6 Allocation Lifecycle

```
Unallocated
→ Paper Shadow
→ Micro Allocation
→ Standard
→ Expanded
→ Reduced
→ Frozen
→ Withdrawn
```

Evidence promotion does not automatically increase capital. Allocation is independently governed and reversible.

Runtime, Security and Execution

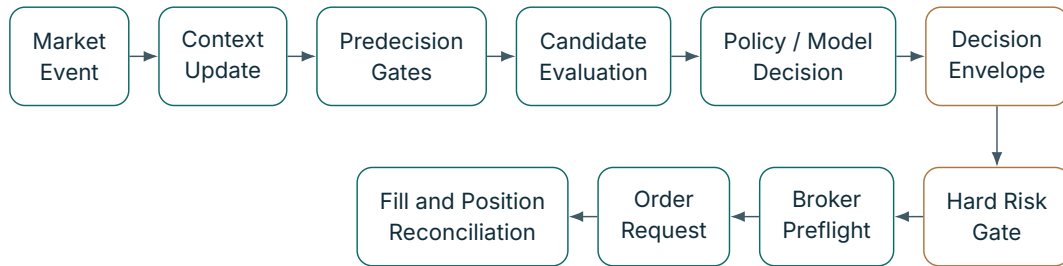


Figure 9.1: The production fast path preserves decision, risk and broker authority as separate layers.

9.1 Runtime Bundle Compiler

Approved research artifacts are compiled into an immutable generation containing:

- ♦ context and feature schema hashes;
- ♦ fixed preprocessing and feature order;
- ♦ model or deterministic policy;
- ♦ calibration and thresholds;
- ♦ complete treatment state machine;
- ♦ risk and allocation limits;
- ♦ abstention and fallback;
- ♦ monitoring contract;
- ♦ signatures and licensing;
- ♦ parity evidence;
- ♦ rollback pointer.

Production never mutates a generation in place.

9.2 Fast Path

```
Market Event
→ Context Update
→ Predecision Gates
→ Candidate Evaluation
→ Model/Policy Decision
→ Utility / Uncertainty / Abstention
→ Decision Envelope
→ Hard Risk Gate
→ Broker Preflight
→ Order Request
→ Fill and Position Reconciliation
```

9.3 Authority Boundaries

The model recommends but cannot place orders, increase size, widen stops or override limits. The execution layer submits and reconciles but cannot rewrite the model decision. The hard risk gate is local, deterministic and fail-closed.

9.4 Hard Risk Gate

Checks include:

- ♦ generation authorization;
- ♦ context freshness and lineage;
- ♦ maximum cash loss;
- ♦ stop and volume validity;
- ♦ account, strategy and portfolio limits;
- ♦ exposure and reservations;
- ♦ daily/weekly loss;
- ♦ margin;
- ♦ broker stop/freeze levels;

- ♦ duplicate/idempotency;
- ♦ market/session permission;
- ♦ kill-switch and incident state.

9.5 Expected Versus Actual

The live ledger separates:

- ♦ expected decision;
- ♦ expected executable price/fill;
- ♦ actual request;
- ♦ actual fill;
- ♦ actual position.

This distinction is essential for diagnosing edge decay versus execution drift.

9.6 Security and Licensing

Security covers:

- ♦ model/code confidentiality;
- ♦ signed artifact integrity;
- ♦ broker/account binding;
- ♦ entitlement and expiration;
- ♦ secrets;
- ♦ tamper detection;
- ♦ update trust;
- ♦ audit retention.

License failure must not make open positions unsafe. Safety-critical exit and risk logic remain operable under a defined degraded mode.

9.7 Incidents and Rollback

Kill levels range from one policy to global entry shutdown. Every generation has a known-good predecessor. The incident flow is:

```
Detect → Contain → Preserve → Reconcile → Diagnose →  
↪ Correct → Revalidate → Resume or Retire
```

Retraining is not an emergency patch; it opens a new governed lineage.

Assumption-Aware Monitoring

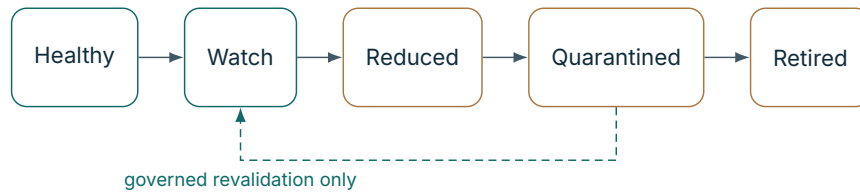


Figure 10.1: Health state changes are explicit, reversible where appropriate, and never driven by an invisible auto-retrain loop.

10.1 Monitoring Thesis

A live strategy is supported by assumptions, not only a historical equity curve. Monitoring therefore asks:

Are the market, model, execution and portfolio assumptions that justified this edge still alive?

10.2 Promotion-Time Monitoring Contract

Each production candidate publishes expected ranges for:

- ♦ context frequency and distribution;
- ♦ features and regime mix;
- ♦ spread, slippage, latency and fill probability;
- ♦ holding time, MFE and MAE;
- ♦ win rate, expectancy and calibration intervals;
- ♦ drawdown and loss clustering;
- ♦ trade coverage and abstention;

- ♦ portfolio dependence and capacity;
- ♦ runtime health.

10.3 Drift Taxonomy

10.3.1 Data Drift

Feed, timestamp, missingness, mapping or specification changes.

10.3.2 Market Drift

Context frequency or market-state distribution changes.

10.3.3 Model Drift

Calibration, ranking, uncertainty or feature relevance deteriorates.

10.3.4 Execution Drift

Costs, latency, fills or broker behavior diverge from assumptions.

10.3.5 Portfolio Drift

Correlation, concentration, capacity or joint drawdown changes.

10.3.6 Edge Decay

The context-treatment-outcome relationship weakens or reverses.

10.4 Health Lifecycle

Healthy
→ Watch
→ Reduced
→ Quarantined
→ Retired

A short period of improved P&L does not automatically reactivate a quarantined policy. Recovery requires governed revalidation.

10.5 Champion–Challenger

New generations run offline, shadow and micro-live before replacing a champion. Comparison includes decision agreement, treatment selection, calibration, latency, execution feasibility and portfolio effect, not only P&L.

10.6 Retraining Governance

The prohibited loop is:

```
Performance down → auto-retrain → silent live  
↪ replacement
```

The correct loop is:

```
Drift alert  
→ reduce/quarantine  
→ diagnostic packet  
→ new experiment  
→ protected validation  
→ challenger generation  
→ shadow comparison  
→ explicit promotion
```

10.7 Closed-Loop Learning

Live fills improve cost models. Failed assumptions create new tests. Drift and retirement update Research Memory. But live outcomes do not automatically rewrite doctrine or production models.

PART VI

Scale, Governance and Delivery

Compounding Speed and Scalability

11.1 What Compounds

Alpha Lab compounds four assets:

1. **Infrastructure** — each shared engine reduces future implementation cost.
2. **Knowledge** — doctrine, decisions and failures prevent repeated reasoning.
3. **Data** — context-treatment outcomes and live observations create proprietary datasets.
4. **Research intelligence** — Meta-Learning and Failure Memory improve experiment selection.

11.2 The Marginal-Cost Shift

Traditional approach:

```
Each new strategy
= signal code
+ data handling
+ backtest
+ management
+ statistics
+ execution
+ monitoring
```

Target Alpha Lab approach:

```
Each new strategy family
≈ doctrine
+ context semantics
+ context engine
+ compatibility declarations
```

The rest is already a platform service.

11.3 Cross-Product Leverage

A new context can use all compatible entries, stops, exits, models, statistical tests, capital policies and runtime engines. A new treatment or trainer becomes available to all compatible contexts.

Therefore one module can create many new research combinations without rebuilding infrastructure.

11.4 Research Memory Leverage

The platform stores why treatments failed, which model families helped, what costs mattered and what regimes were unstable. A new context can start with informed priors instead of uniform brute force.

The Active Experiment Planner can score possible next actions by information gain, economic value, reuse potential, cost, redundancy and false-discovery risk.

11.5 Scalable Dimensions

The architecture scales across:

- ♦ contexts and context compositions;
- ♦ symbols, markets and timeframes;
- ♦ treatment styles;

- ♦ model families and representations;
- ♦ economic scenarios;
- ♦ capital policies;
- ♦ portfolio edges;
- ♦ brokers and data providers;
- ♦ human and AI research agents;
- ♦ compute resources.

11.6 Limits That Do Not Disappear

Engineering speed can become one or more orders of magnitude faster, but prospective proof is still constrained by event frequency, sample maturity, changing regimes and live execution time. The platform accelerates the path to trustworthy evidence; it does not repeal statistical reality.

11.7 Platform KPIs

- ♦ time from doctrine freeze to context conformance;
- ♦ time from context package to first treatment tournament;
- ♦ time from idea to evidence decision;
- ♦ marginal engineering hours per context;
- ♦ percentage of reused modules and generated tests;
- ♦ repeated-hypothesis rate;
- ♦ false-discovery and leakage incidence;
- ♦ prospective conversion rate from candidate to live;
- ♦ evidence and alpha half-life;
- ♦ research value per unit compute and human attention.

11.8 The Real Flywheel

More contexts
→ more outcome data

- better research memory
- better treatment/model priors
- fewer wasted experiments
- faster trustworthy evidence
- more validated edges
- richer live feedback
- better future context research

This is the source of potentially frightening future speed: not faster typing, but permanently decreasing marginal cost and smarter search.

Authority, Lineage and Artifact Chain

12.1 Single-Owner Principle

Every mutable state has one authoritative writer.

Concern	Authority
market meaning	human doctrine owner
context occurrence and lifecycle	context engine
treatment construction	manual policy or treatment compiler
research plan and trial registry	experiment compiler
evidence eligibility	deterministic evidence gate
explanatory interpretation	AI analyst, non-authoritative
capital limits	capital and portfolio governance
broker action	execution layer after risk gate
live activation	deployment governance
emergency shutdown	hard risk/operator authority

12.2 Canonical Artifact Chain

Doctrine

→ ContextSpec

→ ContextEvents

→ ContextSnapshots

```
→ TreatmentInventory
→ Outcomes
→ Dataset + Folds
→ TrialLedger + Predictions
→ Statistics + AntiOverfit
→ AnalystReport
→ EvidenceDecision
→ CapitalPolicy
→ PortfolioAdmission
→ RuntimeGeneration
→ Decision / Order / Fill Ledger
→ Monitoring / Incident / Retirement
```

12.3 Required Identity Chain

Every final claim binds:

- ♦ source inventory;
- ♦ code and build;
- ♦ doctrine/context versions;
- ♦ schema and feature order;
- ♦ treatment and cost profile;
- ♦ dataset and fold plan;
- ♦ experiment and trial;
- ♦ model, calibration and threshold;
- ♦ evidence policy;
- ♦ capital/portfolio policy;
- ♦ runtime generation;
- ♦ approvals.

A result that cannot be reconstructed through this chain is exploratory only.

12.4 Versioning

Major version changes meaning or compatibility. Minor adds backward-compatible capability. Patch fixes wording or defects without semantic reclassification.

When a context definition changes historical classification, prior evidence remains attached to the old lineage.

12.5 Promotion and Waivers

Promotion gates cover context, research integrity, statistical evidence, economics, prospective parity, portfolio admission and production readiness.

A waiver may temporarily relax a noncritical requirement but must identify scope, reason, tighter controls, expiry and remediation. Critical leakage, artifact corruption and hard risk limits cannot be waived.

12.6 Audit Questions

The architecture must answer:

- ♦ What did the system know at decision time?
- ♦ Which doctrine created the context?
- ♦ Which alternatives were searched?
- ♦ Which data selected the winner?
- ♦ Why was it promoted?
- ♦ What risk was requested and approved?
- ♦ What order was expected and filled?
- ♦ Which assumption changed before reduction or retirement?

Implementation Program

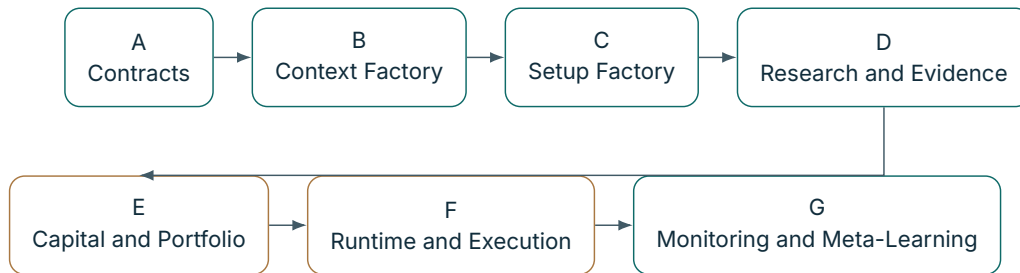


Figure 13.1: Implementation sequence for one golden vertical slice and all reusable mechanisms encountered along the way.

13.1 Delivery Strategy

Build one complete real vertical slice while centralizing every reusable mechanism encountered. Avoid:

- ♦ an abstract platform with no real context;
- ♦ separate strategy stacks that repeat infrastructure.

13.2 Phase A — Canonical Contracts

Deliver IDs, lineage, market event, context, treatment, outcome, dataset, trial, evidence, capital and runtime schemas. Prove one synthetic opportunity can travel end to end.

13.3 Phase B — Context Factory

Deliver Market Truth, Context SDK/compiler, lifecycle framework, reusable primitives, scenario/replay harness and one golden context with cross-mode parity.

13.4 Phase C — Setup Factory

Deliver manual policy contract, treatment atoms, compatibility graph, objective profiles, abstention, trainer SDK, baseline ladder and manual/AI policies for the golden context.

13.5 Phase D — Research and Evidence

Deliver experiment compiler, exposure/trial ledger, outcome/economics, dataset/folds, statistics, Monte Carlo, anti-overfit, null generator, evidence states, AI Analyst and Failure Memory.

13.6 Phase E — Capital and Portfolio

Deliver money-management policies, separate capital validation, edge genome, portfolio dependence, allocation, reservation and capacity.

13.7 Phase F — Runtime, Security and Execution

Deliver runtime compiler, fixed fast path, decision envelope, hard risk gate, broker adapters, reconciliation, licensing, telemetry, kill and rollback.

13.8 Phase G — Monitoring and Meta-Learning

Deliver machine-readable assumptions, drift and health, champion/challenger, governed retraining, Research Memory, meta-features and Active Experiment Planner.

13.9 Golden Vertical Slice Definition

The slice is complete only when it covers:

```
Doctrine
→ Context
→ Human Setup
→ AI Tournament
→ Outcome
→ Protected Evidence
→ Capital
→ Portfolio
→ Runtime
→ Paper/Shadow
→ Monitoring
```

It is considered successful even if every setup is rejected, provided the rejection is trustworthy, reproducible and becomes reusable failure knowledge.

13.10 Development Rhythm

Use alternating cycles:

```
Expansion cycle
→ build capability and discover domain requirements

Consolidation cycle
→ unify contracts, delete duplication, strengthen
↔ tests and automate artifacts
```

This preserves speed without letting architectural entropy erase the compounding effect.

Architectural Risks and Controls

14.1 False Discovery From Search Scale

14.1.1 Risk

AI can test enough setups to find spectacular noise.

14.1.2 Controls

Complete trial ledger, hierarchical multiple-testing corrections, PBO/deflated metrics, nulls, stability tests and prospective evidence.

14.2 Test-Set Feedback Contamination

14.2.1 Risk

AI Analyst or human redesign based on final-test results turns test into training.

14.2.2 Controls

Research Firewall, exposure ledger, immutable data roles and new lineage after exposure.

14.3 Context Redefinition After Results

14.3.1 Risk

The context is edited until it becomes profitable.

14.3.2 Controls

Doctrine freeze, semantic versioning, migration and separate evidence lineage.

14.4 Metric Gaming

14.4.1 Risk

High win-rate creates tiny targets and huge tail risk; high reward creates unusably sparse trades.

14.4.2 Controls

Multi-objective constrained optimization, sample/coverage guardrails, tail and cost limits.

14.5 Money-Management Overfit

14.5.1 Risk

Sizing rules are optimized on the same history and appear to improve edge.

14.5.2 Controls

Separate capital research and validation, simple baselines, Monte Carlo and portfolio tests.

14.6 Strategy-Specific Duplication

14.6.1 Risk

Every context builds its own outcome, statistics, execution and monitoring.

14.6.2 Controls

Plugin contracts, Context SDK, treatment compiler, shared evidence and golden-slice discipline.

14.7 Over-Generalization

14.7.1 Risk

A universal engine erases domain meaning.

14.7.2 Controls

Modularize mechanisms, not semantics. Context-specific doctrine and transition rules remain first-class.

14.8 Silent Live Adaptation

14.8.1 Risk

Auto-retraining or threshold drift mutates production.

14.8.2 Controls

Immutable runtime generations, shadow/challenger, explicit promotion and rollback.

14.9 AI Analyst Hallucination

14.9.1 Risk

Convincing explanation becomes accepted causal truth.

14.9.2 Controls

Artifact-grounded claims, confidence, counter-evidence, falsification proposals and non-authoritative status.

14.10 Portfolio Blindness

14.10.1 Risk

Individually valid strategies create concentrated joint exposure.

14.10.2 Controls

Edge genome, shared-event clusters, exposure mapping, reservations and portfolio risk authority.

14.11 Security Breaking Safety

14.11.1 Risk

License or remote-service failure prevents safe management of open trades.

14.11.2 Controls

Degraded safe mode, local hard risk, independent kill and safe exit capability.

14.12 Documentation Becoming Detached

14.12.1 Risk

Documents become aspirational and code diverges.

14.12.2 Controls

Machine-readable specs, generated manifests, executable fixtures, contract tests and linked artifacts.

14.13 Complexity Overwhelming Speed

14.13.1 Risk

The platform becomes harder to understand than rebuilding strategies.

14.13.2 Controls

Golden paths, clear ownership, module KPIs, deletion/deprecation cycles, automated artifact generation and measurement of marginal context cost.

Acceptance Gates and Completion Criteria

A.1 Context Gate

- ♦ deterministic replay;
- ♦ known-time audit;
- ♦ frozen feature schema;
- ♦ stable cluster IDs;
- ♦ treatment allowlist;
- ♦ manual baseline.

A.2 Treatment Gate

- ♦ buy/sell golden fixtures;
- ♦ cost and risk monotonicity;
- ♦ broker normalization;
- ♦ reachable state machine;
- ♦ fail-closed compatibility.

A.3 Training Gate

- ♦ trainer contract;
- ♦ immutable manifest;
- ♦ determinism or tolerance evidence;
- ♦ complete OOF predictions;

- ♦ locked dependencies;
- ♦ export eligibility.

A.4 Statistical Gate

- ♦ clean leakage audit;
- ♦ cluster-safe folds;
- ♦ multiple-testing policy;
- ♦ confidence intervals;
- ♦ PBO and deflated performance;
- ♦ stress and concentration tests;
- ♦ economic relevance.

A.5 Runtime Gate

- ♦ feature and preprocessing parity;
- ♦ Python/MQL5 prediction parity;
- ♦ bounded latency;
- ♦ cost/risk identity match;
- ♦ fail-closed tests;
- ♦ no model-to-broker authority.

A.6 Promotion Gate

- ♦ complete evidence;
- ♦ human approval;
- ♦ prospective paper period;
- ♦ acceptable shadow divergence;
- ♦ micro-live allowlist and kill switch;
- ♦ rollback drill;
- ♦ monitoring baseline.

A.7 Cross-Mode Conformance Matrix

Concern	Historical research	Strategy Tester	Paper/shadow	Micro-live/live
Source availability	frozen inventory	terminal history snapshot	observed stream	observed stream
Time authority	event/known-time ledger	tester clock	broker/UTC clock	broker/UTC clock
Economic truth	replayed bid/ask and costs	tester plus reconciled overrides	observed quote/fill model	broker checks and transactions
Decision artifact	immutable prediction ledger	immutable result sink	shadow/paper ledger	live decision and transaction ledger
Failure response	reject experiment row/run	fail test or abstain	abstain/reconcile	no-send, management-only, kill switch, rollback

A.8 Definition of Complete

- ☐ The implementation is deterministic under identical inputs and exact versions.
- ☐ All causal and economic assumptions are machine-readable.
- ☐ Manual-only, simple baseline, and AI-assisted paths are comparable on identical opportunities.
- ☐ Every attempted model, treatment, threshold, subgroup, and ensemble is present in the selection ledger.
- ☐ Anti-overfit, stress, parity, capacity, prospective, and operational gates are passed or explicitly rejected.
- ☐ No model output can bypass hard risk, broker, authorization, or kill-switch constraints.

Canonical Object Catalogue

B.1 Knowledge and Context Objects

- ♦ **DoctrinePackage:** versioned market meaning, vocabulary, examples, ambiguities, lifecycle and decision history.
- ♦ **ContextSpec:** machine-readable context contract covering fields, identity, known-time, lifecycle, compatibility and missingness.
- ♦ **ContextOccurrence:** immutable occurrence emitted by the deterministic context engine.
- ♦ **ContextSnapshot:** decision-time representation of one occurrence with fixed feature order and source lineage.
- ♦ **ContextPackage:** publishable bundle containing doctrine, specification, adapters, features, representations, allowed treatments, labels, monitoring assumptions and conformance fixtures.

B.2 Setup and Treatment Objects

- ♦ **SetupThesis:** falsifiable human or AI exploitation hypothesis bound to explicit context versions.
- ♦ **TreatmentAtom:** one entry, stop, exit, management, sizing or execution primitive.
- ♦ **CompiledTreatment:** complete, coherent and broker-feasible policy with eligibility and abstention.
- ♦ **ObjectiveProfile:** constrained preferences such as high win rate, high reward, low drawdown, runner or high frequency.
- ♦ **PolicyGraph:** explicit composition of manual rules, model outputs, filters and hard gates.

B.3 Research and Evidence Objects

- ♦ **OutcomeRecord:** path-based result including entry, stop, targets, MFE, MAE, duration and causal timestamps.
- ♦ **EconomicScenario:** executable bid/ask, commission, slippage, latency, fill and capacity assumptions.
- ♦ **DatasetManifest:** source inventory, context/treatment lineage, label policy, folds and transformations.
- ♦ **ExperimentPlan:** immutable compiled hypothesis, search universe, budget, metrics, stopping and evidence criteria.
- ♦ **TrialLedger:** complete record of completed, failed, pruned, duplicate, invalid, cancelled and timed-out attempts.
- ♦ **EvidenceDecision:** deterministic state transition supported by registered statistics, stress, parity and governance artifacts.
- ♦ **AnalystReport:** non-authoritative explanation of profit, loss, regime, cost and alternative hypotheses.

B.4 Capital and Production Objects

- ♦ **CapitalPolicy:** versioned risk function, floors, ceilings, drawdown response, reserve requirement and kill conditions.
- ♦ **EdgeGenome:** reusable profile of market, horizon, treatment style, return shape, regime, capacity, costs, dependence and evidence state.
- ♦ **PortfolioAdmission:** portfolio-level decision covering dependence, concentration, reservations, capacity and priority.
- ♦ **RuntimeGeneration:** immutable package of schemas, preprocessing, model/policy, thresholds, treatment, risk, monitoring, signatures and rollback.
- ♦ **DecisionEnvelope:** exact decision-time context, candidate set, utility, uncertainty, abstention, requested risk and lineage.
- ♦ **ExecutionLedger:** expected decision, expected fill, actual request, actual fill, position and reconciliation history.
- ♦ **MonitoringContract:** machine-readable assumptions, expected ranges, alert thresholds, actions and revalidation requirements.

Platform KPI Catalogue

C.1 Speed and Reuse

- ♦ Time from doctrine freeze to context conformance.
- ♦ Time from a conformed context package to the first treatment tournament.
- ♦ Time from hypothesis registration to an evidence decision.
- ♦ Marginal engineering hours required for each new context.
- ♦ Percentage of generated tests, reused mechanisms and shared services.
- ♦ Repeated-hypothesis rate and duplicated infrastructure rate.

C.2 Research Integrity

- ♦ Leakage and protected-data exposure incidents.
- ♦ Complete-trial-ledger coverage.
- ♦ False-discovery rate after prospective evidence.
- ♦ Percentage of candidates rejected by semantic, causal, economic, statistical and operational gates.
- ♦ Final-test consumption rate per successful production lineage.

C.3 Production Quality

- ♦ Historical, tester, paper, shadow and live parity.
- ♦ Decision latency and timeout rate.
- ♦ Order rejection, slippage, partial-fill and reconciliation error rates.
- ♦ Runtime generation rollback and recovery time.
- ♦ Evidence half-life, alpha half-life and drift-to-action latency.

C.4 Economic Productivity

- ♦ Research value per unit of compute and human attention.
- ♦ Prospective conversion rate from candidate to paper, micro-live and production.
- ♦ Capital efficiency after costs, dependence and capacity constraints.
- ♦ Portfolio diversification contribution of newly admitted edges.

Source Library Map

This master architecture synthesizes and organizes four canonical libraries included in the vault.

D.1 Strategy Factory

Canonical entry point: Strategy Factory master index.

Best used for:

- ♦ governance and strategy lifecycle;
- ♦ anatomy adapter contract;
- ♦ candidate universe;
- ♦ outcome/statistical packs;
- ♦ anti-overfit suite;
- ♦ model training;
- ♦ execution;
- ♦ portfolio and promotion;
- ♦ templates and runbooks.

D.2 Strategy Factory V2

Canonical entry point: Strategy Factory V2 master index.

Best used for:

- ♦ fast-path architecture;
- ♦ modular kernel and plugins;
- ♦ context DAG and caching;
- ♦ precompiled candidate templates;

- ♦ utility, uncertainty and abstention;
- ♦ latency, concurrency and atomic generation;
- ♦ validation and chaos testing;
- ♦ V1-to-V2 migration.

D.3 Universal Context Exploitation Engine

Canonical entry point: Universal Context Exploitation Engine master index.

Best used for:

- ♦ Context SDK;
- ♦ opportunity/treatment decomposition;
- ♦ manual setup as first-class policy;
- ♦ trainer plugin architecture;
- ♦ treatment atom registries;
- ♦ dataset and label taxonomy;
- ♦ algorithm families;
- ♦ anti-overfit and model selection;
- ♦ implementation program and real context tournament.

D.4 AI Algorithm Engineering OS

Canonical entry point: AI Algorithm Engineering OS home.

Best used for:

- ♦ doctrine and formalization workflow;
- ♦ algorithm and state-machine design;
- ♦ architecture standards;
- ♦ implementation and patch protocol;
- ♦ testing and verification;
- ♦ AI operating roles;
- ♦ Obsidian knowledge management;
- ♦ MQL5 specialization;
- ♦ templates, checklists and governance.

D.5 Reading Order

1. Read the master architecture folder.
2. Use the source libraries only when implementing or reviewing a specific layer.
3. The master architecture owns overall system flow; source libraries own deep detailed mechanisms.

Glossary

Anatomy: deterministic structural interpretation of market data.

Context: immutable causal state derived from anatomy and shared services.

Opportunity: directional possibility before selecting a trade treatment.

Treatment: entry, stop, exit, management, sizing, cost, and execution combination.

Manual policy: deterministic user-authored exploitation rule.

Model policy: decision rule using a governed model artifact.

Hybrid policy: explicit graph combining manual rules and model outputs.

Known time: earliest timestamp when a field could have been used.

Label maturity: timestamp when an outcome becomes fully observable.

Cluster: related rows that must stay together for splitting or resampling.

Trial: one immutable hypothesis variation.

Treatment regret: selected treatment utility minus best feasible treatment utility.

PBO: probability of backtest overfitting.

Deflated performance: performance adjusted for search and non-normality.

Abstention: explicit no-action decision.

Runtime bundle: immutable model, preprocessing, policy, compatibility, and parity package.